

UPS REPRICE PLATFORM

Supabase 完整說明書

白話解釋 Supabase 是什麼、能幫你做什麼

你現在這套系統的完整架構圖

一步一步:在 Supabase 架設帳號密碼

專案:sandyliu3056/ups-reprice-web

分支:claude/supabase-setup-auth-0px1az

製作日期:2026 年 8 月 28 日

內容依據:此 repo 目前實際的程式碼與設定檔

四個章節,可以分開看。如果你只想趕快把登入弄好,直接翻到第 4 章;但第 3 章最後那一頁的警告請務必先看過。

第 1 章 Supabase 是什麼(全部白話)

一句話比喻、為什麼你的網站需要它、五個零件、三把鑰匙、費用

第 2 章 我可以拿它幹嘛

已經在用的 3 件事、打開開關就有的 3 件事、以後可以做的 4 件事、它做不到的事

第 3 章 你現在的系統架構

全景圖、登入流程對照圖、資料放在哪裡、檔案地圖、兩條產品線

第 4 章 一步一步:架設帳號密碼

12 個步驟,含所有要貼的 SQL 與設定;最後有排錯對照表

先講一個最重要的事實

你的網站現在的登入,其實沒有走 Supabase。目前是「本機帳號模式」——帳號和密碼的雜湊值寫在 `index.html` 裡面。Supabase 現在只被拿來當「跨電腦同步設定和帳單歷史」的置物櫃而已。

整套 Supabase 登入的程式碼都已經寫好了,就躺在檔案裡,只是開關沒有打開。第 4 章就是在教你怎麼把它打開。

1 Supabase 是什麼(全部白話)

■ 一句話

Supabase 是「租來的後端」。

你的工具現在是一份網頁(`index.html`),它是一間店面—— 客人走進來、東西在店裡處理完、拿了就走。
Supabase 是店面後方的 倉庫、櫃檯、和保全:它幫你存東西、幫你查驗誰能進來、幫你保管 不能給客人看到的鑰匙。

重點是「租」:你不用買伺服器、不用架資料庫、不用管備份和更新。 註冊一個帳號,五分鐘後就有一整套後端可以用,而且這個規模的用量是免費的。

■ 為什麼你的網站會需要它

你的工具現在所有事情都發生在使用者自己的瀏覽器裡。這種做法有很大的好處—— 帳單和費率不會離開那台電腦,速度也快——但有三件事永遠做不到:

做不到的事	為什麼	Supabase 怎麼補
密碼沒有地方放	網頁裡的任何東西,按 F12 就看得到。就算存的是雜湊值不是明碼, 那道鎖也是掛在門外面的——擋君子不擋小人。	密碼交給 Supabase 的 Auth 保管。瀏覽器從頭到尾拿不到密碼, 只拿到一張有時效的識別證(token)。
換一台電腦,東西就不見	設定、帳單歷史都存在那台瀏覽器裡。換電腦、清快取、換瀏覽器, 全部歸零。	存進 Supabase 的資料庫。人在哪台電腦登入,資料就跟到哪台。
API 金鑰不能放前端	UPS 的地址驗證要 Client Secret。放進網頁等於公開, 別人可以拿去用你的額度。	金鑰放在 Supabase 的 Edge Function 裡。瀏覽器只丟地址進去、 拿結果回來,永遠看不到金鑰。

■ Supabase 的五個零件

Supabase 不是一個東西,是五個服務打包在一起。你不用全部都用—— 你這個專案目前只用到其中三個。

零件	白話比喻	你這個專案的用途	現況
Auth 認證	大樓一樓的櫃檯,查證件、發識別證,識別證過期會自動換新的	登入頁的帳號密碼	程式已寫好 尚未啟用
Database Postgres 資料庫	一排檔案櫃,每個抽屜裝一種資料	設定同步、帳單歷史、登入紀錄、帳號清單	使用中
RLS 列級安全	每一份文件自己的鎖,不是整個櫃子一把鎖	只有主帳號讀得到登入紀錄;每個人只看得到自己那一列設定	使用中

零件	白話比喻	你這個專案的用途	現況
Edge Functions	你在雲端請的一位小助理,幫你做「不能讓客人看到過程」的事	<code>ups-address</code> (用 UPS 金鑰查住宅/商業地址)、 <code>admin-users</code> (用管理員權限開帳號)	程式已寫好 看你部署了沒
Storage 檔案儲存	雲端硬碟	目前沒用到(帳單檔不上雲,這是刻意的)	未使用

■ 三把鑰匙,千萬不要搞混

這是 Supabase 唯一真的會出事的地方,值得花兩分鐘看懂。Supabase 會給你 好幾組看起來很像的字串,但它們的權力天差地遠:

名稱	可以放哪裡	它能做什麼	外流的後果
Project URL <code>https://xxx.supabase.co</code>	前端(<code>auth-config.js</code>)	只是一個地址	沒事。它本來就是要公開的
anon key (新版叫 publishable key)	前端(<code>auth-config.js</code>)	只能做 RLS 規則允許的事。規則寫好了,它就是安全的	沒事。設計上就是要放進瀏覽器的
service_role key (新版叫 secret key)	只能放 Edge Function 的環境變數。絕對不能進 repo、不能進任何 <code>.js</code> 、不能貼給任何人	繞過所有 RLS。整個資料庫任它讀、任它改、任它刪	災難。 所有資料全開,而且它不會過期

你的 repo 已經有一道防線

`.github/pre-commit` 會擋住看起來像 JWT 或 Supabase secret key 的字串,不讓它被 commit 進去。記得裝上它:

```
git config core.hooksPath .github
```

但別靠它。那是安全網,不是規則本身。

■ 要花多少錢

這個規模的工具,免費方案綽綽有餘。Supabase 免費方案大致提供 500 MB 資料庫、每月數萬名活躍使用者、1 GB 檔案儲存(實際數字以官方定價頁為準,他們偶爾會調整)。你這個工具的整個資料庫大概就是幾張小表,連 1 MB 都用不到。

免費方案唯一要注意的一件事

免費專案如果連續約一週完全沒有任何連線,會被自動暫停(paused)。暫停之後網站的登入就會失敗。

解法很簡單:進 Supabase 後台按一下「Restore / Resume」就恢復,資料不會掉。如果你的工具是天天在用的,根本不會碰到這件事。

2 我可以拿它幹嘛

分成四組:A 現在已經在用的、B 程式寫好了只差開關的、C 以後想做可以做的、D 不要拿它做的。

■ A. 現在已經在用的(3 件)

A1 — 跨電腦同步你的費率設定

你在公司存好的費率設定,回家用另一台電腦、同一組帳號密碼登入,設定會自己跟過來。

做法很巧妙:瀏覽器在你登入的當下,用 `sha256("cfg-sync::" + 帳號 + "::" + 密碼)` 算出一把「同步鑰匙」,設定就存在那把鑰匙底下。資料庫那張表沒有任何 RLS policy——意思是任何人拿 anon key 直接讀寫都會被擋死,唯一的入口是兩個函式,而那兩個函式一定要你交出完整的 64 位鑰匙才給你那一列,而且沒有任何「列出清單」的路。

就像一個憑密碼取物的置物櫃:不知道密碼就算不出鑰匙,也不知道櫃子裡有幾格。

副作用要知道:改密碼就等於換一把新鑰匙。舊密碼底下存的設定不會跟過來——改完密碼記得再按一次「儲存設定」。

A2 — 跨電腦同步帳單歷史

同一把鑰匙,另一張表。每一期歸檔的帳單會在瀏覽器裡先 gzip 壓縮再上傳;換一台電腦登入,那台電腦沒有的期別就會被抓下來。

規則是只增不減:在某台電腦上清掉瀏覽器資料,不會把雲端那份刪掉。這是刻意的——歷史帳單不該因為某個人清了快取就消失。

A3 — UPS 地址驗證(住宅 / 商業判定)

住宅和商業的附加費差很多,判錯就算錯。UPS 有一支 API 可以查,但瀏覽器不能直接打——需要 OAuth 金鑰,而且會被 CORS 擋。

`supabase/functions/ups-address` 這支 Edge Function 就是中間人:金鑰放在它的環境變數裡,前端只丟地址進去,拿回 R(住宅)/C(商業)/U(判不出)。

設定的地方在「帳單歷史」頁的 API 端點欄,填 `https://<你的專案>.supabase.co/functions/v1/ups-address`。

■ B. 程式已經寫好,只差把開關打開(3 件)

以下三件事的程式碼全部都在 repo 裡了,只是因為現在跑在「本機帳號模式」所以沒有生效。第 4 章教的就是怎麼打開。

B1 — 真正的帳號密碼登入

密碼由 Supabase 保管,不再寫在 `index.html` 裡。好處:

- 要加人、要停權、要改密碼,不用改程式、不用重新部署——在後台點一點就好。

- 密碼不會出現在 repo 裡,也不會留在 Git 歷史裡。
- 登入狀態是有時效的 token, Supabase 會自動換新;離職的人一停權就真的進不來。
- 可以開「忘記密碼」自動寄信重設(前提是那個帳號用的是真的 email)。

B2 — 在網站裡直接開帳號、改角色、刪帳號

Admin 分頁裡的帳號管理。新增帳號需要 `service_role` 權限,而那把鑰匙 絕對不能進瀏覽器,所以做法是:網頁呼叫 `admin-users` 這支 Edge Function, 函式先驗來人是不是主帳號(檢查 token 裡的 `app_metadata.role`), 驗不過直接 403,連你送了什麼都不看。

現在這個分頁在本機模式下只能「看」,不能改——因為帳號寫在 `index.html` 裡, 要加人得改檔案再部署一次。

B3 — 登入紀錄(誰、什麼時候、用什麼裝置)

現在的「登入紀錄」只看得到這一台瀏覽器的紀錄——同事在他自己電腦上登入, 你這邊看不到。切到 Supabase 之後,紀錄寫進資料庫,主帳號可以看到所有人的。

寫入的方式也是安全的:瀏覽器不能直接寫那張表(權限被收掉了), 只能請一個函式「幫我記一筆」,而那個函式的使用者 ID、email、時間全部是伺服器 自己從已驗證的 token 裡取的,不是瀏覽器送什麼就記什麼。

■ C. 以後想做可以做的(4 件)

想做的事	怎麼做	準備到哪了
每個帳號自己的費率存雲端	切到 Supabase 模式後自動生效,存在 <code>user_settings</code> 表,一個帳號一行	<code>supabase/schema.sql</code> 已經寫好
給客戶一個唯讀連結	開一張只讀的表 + 一條 RLS 規則,客戶用連結看自己那期的報表,看不到成本和毛利	還沒做,但架構撐得住
忘記密碼自動重設	Supabase Auth 內建。但帳號要是真的 email 才收得到信	開關在後台,免費
帳單原始檔存雲端	用 Storage。但先想清楚要不要——現在「檔案不離開電腦」本身就是一個賣點	未使用

■ D. 不要拿 Supabase 做的事

- 它不會幫你算費率。那六萬行的計價引擎在瀏覽器裡跑,不上雲。這是好事:你的計價邏輯是你的資產,不用交給任何人代管,而且大帳單在本機算比較快。
- 不要把 `service_role` 金鑰放進前端,任何理由都不行。 需要高權限的事,一律做成 Edge Function。
- 它不能保護「已經送到瀏覽器的畫面」。private repo 保護的是原始碼; 一個已經登入的人,永遠可以按 F12 看到頁面上的所有東西。要真正藏起來的東西,就不能送到前端。

3 你現在的系統架構

這一章畫的是現況,不是理想狀態。看得懂這兩張圖,就知道第 4 章那些步驟到底在動什麼。

圖 1:全景圖

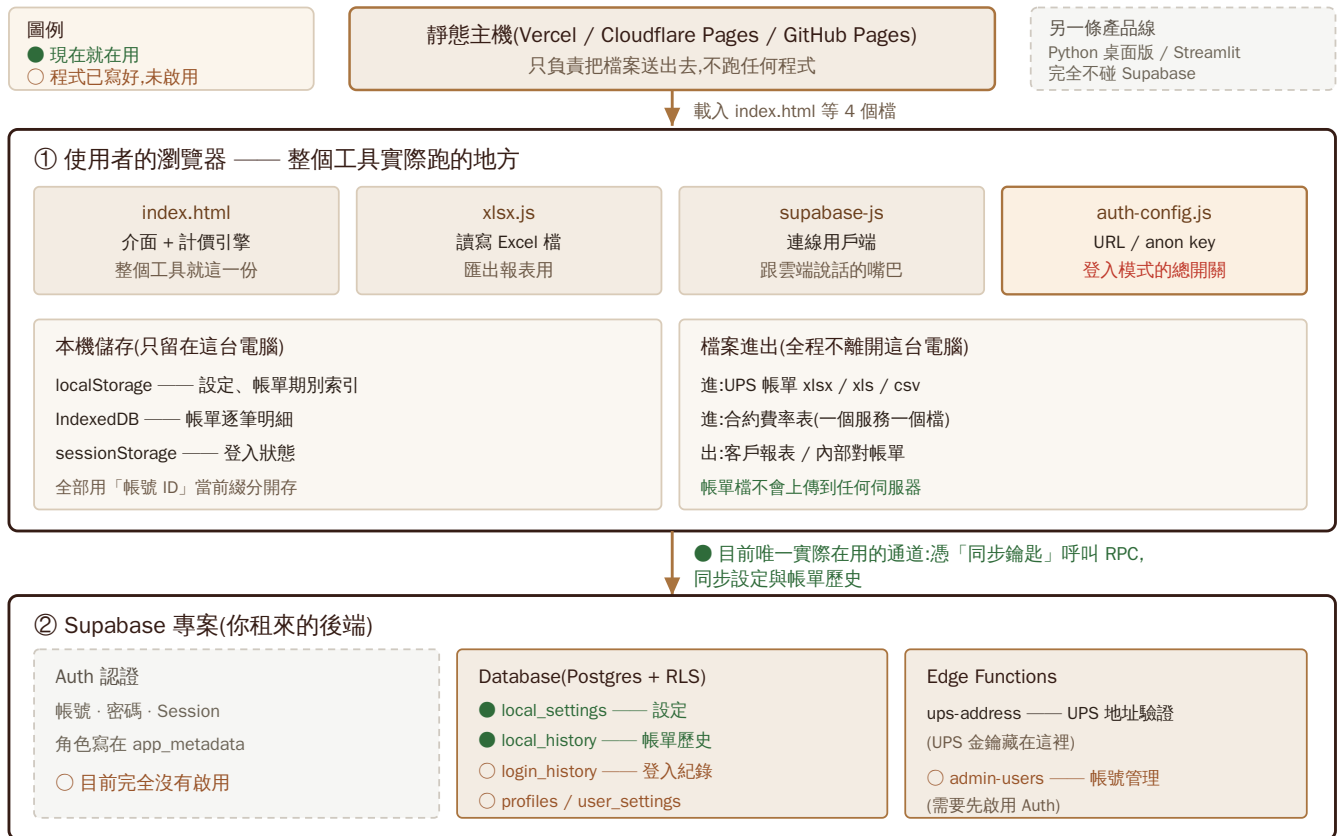


圖 1 — 現況全景。綠色是現在真的在跑的路;灰色虛線和 ○ 是程式已經寫好、開關還沒打開的部分。

圖 2:登入流程,現在 vs 切換後



圖 2 — 兩種登入模式。決定走哪一邊的,只有 auth-config.js 裡有沒有 authMode: "supabase" 這一行。

3 你現在的系統架構(續)

■ 現況的三個重點

重點一:登入沒有走 Supabase

`auth-config.js` 裡沒有 `authMode: "supabase"` 這一行, 所以程式判定為本機帳號模式。帳號和密碼的雜湊值寫在 `index.html` 的 `LOCAL_ACCOUNTS` 裡, 登入時在瀏覽器裡比對。

要換人、加人、改密碼, 現在都得改 `index.html` 再重新部署一次。

重點二: Supabase 現在只當「置物櫃」

雖然沒有登入, 但設定和帳單歷史還是有存雲端——用的是一把從「帳號 + 密碼」算出來的同步鑰匙, 走兩個特製的資料庫函式。這是為了在還沒接 Auth 之前, 先讓換電腦不要掉資料。

重點三: repo 裡有一個沒有被用到的檔案

`index.html` 載入的是 `auth-config.js`(有連字號)。repo 裡另外有一個 `authconfig.js`(沒有連字號)——沒有任何地方載入它。改那個檔不會有任何效果。

建議挑一個時機把它刪掉, 免得以後有人改錯檔案、找半天找不到原因。

■ 你的資料現在存在哪裡

哪一種資料	存在哪	換電腦還在嗎	誰看得到
帳號與密碼	<code>index.html</code> 的 <code>LOCAL_ACCOUNTS</code> (存的是雜湊, 不是明碼)	在(寫在程式裡)	任何打得開網頁的人都看得到那串雜湊
登入狀態	<code>sessionStorage</code> ; 勾了「保持登入」才會寫進 <code>localStorage</code>	否	本人
合約費率設定	<code>localStorage</code> (快取) + Supabase <code>local_settings</code>	會跟過去(同一組帳號密碼)	算得出同步鑰匙的人, 也就是知道密碼的人
帳單期別索引	<code>localStorage</code> + Supabase <code>local_history</code> (gzip 壓縮)	會跟過去	同上
帳單逐筆明細	<code>IndexedDB</code> (本機)	要看有沒有同步過那一期	本機
上傳的帳單 / 費率檔	只在記憶體裡, 算完就沒了	否	沒有人。檔案不上傳
登入紀錄	本機的活動紀錄(<code>localStorage</code>)	否	只看得到這一台的紀錄

■ 檔案地圖

檔案	是什麼	要不要上傳到網站
<code>index.html</code>	整個工具:介面 + 計價引擎	要
<code>auth-config.js</code>	登入設定:URL、anon key、模式開關	要
<code>supabase-js-2.112.3.js</code>	Supabase 用戶端	要
<code>xlsx-0.18.5.full.min.js</code>	讀寫 Excel	要
<code>_headers</code> / <code>vercel.json</code> / <code>.nojekyll</code> / <code>web.config</code> / <code>nginx.conf.example</code>	各家主機的快取設定,避免瀏覽器拿到舊版	依你用哪家主機挑一個
<code>supabase/schema.sql</code>	建 profiles / user_settings 兩張表	不上傳。在 Supabase SQL Editor 跑一次
<code>supabase/login_history.sql</code>	建登入紀錄表 + 安全的寫入函式	同上
<code>supabase/local_settings.sql</code>	建同步用的表與兩個 RPC 函式	同上(這支已經跑過了,同步才會通)
<code>supabase/functions/admin-users/</code>	帳號管理(需要 service_role)	部署成 Edge Function
<code>supabase/functions/ups-address/</code>	UPS 地址驗證代理	部署成 Edge Function
<code>authconfig.js</code>	沒有被任何地方載入的舊檔	不需要,建議刪掉
<code>UPS_Reprice_Tool.py</code> / <code>engine.py</code> / <code>app.py</code>	Python 桌面版與 Streamlit 版	不上傳,跟網頁版無關
<code>.githooks/pre-commit</code>	擋住金鑰被 commit 進去的安全網	裝在本機: <code>git config core.hooksPath .githooks</code>

■ 兩條產品線,不要搞混

	網頁版	Python 版
主檔案	<code>index.html</code> (一份就是全部)	<code>UPS_Reprice_Tool.py</code> + <code>engine.py</code> + <code>app.py</code>
怎麼跑	開網址就好,不用安裝	桌面程式,或 <code>streamlit run app.py</code>
登入	有(本機模式 / Supabase 模式)	沒有
用到 Supabase 嗎	會	完全不會
計價邏輯	兩邊是同一套規則。Python 版的 <code>engine.py</code> 直接呼叫桌面版的引擎,不另外抄一份——就是為了避免「網頁版算出來的數字跟桌面版不一樣」	

4 一步一步:在 Supabase 架設帳號密碼

12 個步驟。前 9 步都在 Supabase 後台點,不用碰程式;第 10 步才改一個檔案的三行字。第 12 步請務必先讀完再開始。

0 開始前先確認三件事

- 你有一個能收信的 email(註冊 Supabase 用)。
- 你知道網站現在部署在哪(Vercel?Cloudflare Pages?GitHub Pages?), 而且改了檔案之後你有辦法重新部署。
- 先把現在的設定匯出成檔案存好。工具裡「設定」頁有匯出功能——這是第 12 步的保險,做了不吃虧。

1 建立 Supabase 專案

到 supabase.com → 註冊(可以直接用 GitHub 帳號登入) → New project。要填三樣:

- Name:隨便取,例如 `ups-reprice`。
- Database Password:這是資料庫本身的密碼,跟等一下要建的使用者帳號密碼完全是兩回事。存進密碼管理器就好,平常用不到。
- Region:挑離使用者最近的。台灣選 Singapore 或 Tokyo, 人在美國就選 US West / US East。

按下去之後大約等一到兩分鐘,專案才會建好。

如果你已經有專案了(你的 `auth-config.js` 裡已經填了一組 URL 和 key,表示你有), 這一步跳過,直接從第 2 步確認一下就好。

2 拿到 Project URL 和 anon key

左側選單最下面的齒輪 Project Settings → API (新版可能叫 API Keys)。抄兩個東西:

- Project URL,長得像 `https://xxxxxxx.supabase.co`
- anon / public key(新版叫 publishable key)

同一頁上還有一個 service_role(或 secret)key。那一個不要抄,不要複製,不要貼到任何地方。它會繞過所有安全規則。只有第 8 步的 Edge Function 用得到它,而那個是 Supabase 自己傳給函式的,你根本不用經手。

3 關掉公開註冊 —— 這一步最重要

Authentication → Sign In / Providers → 點 Email → 把 Allow new users to sign up 關掉(有些版本寫 Enable Sign Ups)→ Save。

為什麼一定要關:Supabase 預設是開放註冊的。不關的話,任何知道你網址的人 都可以自己註冊一個帳號走進來,看你的工具。關掉之後,登入頁還是誰都打得開,但只有你親手開的帳號進得去。

4 建立第一個帳號(你自己的主帳號)

Authentication → Users → Add user → Create new user。

三個欄位:

- Email:見下面那張表,格式很重要。
- Password:設一個新的,不要沿用以前寫在程式裡的那組——那些已經在 Git 歷史裡了,等於公開。
- Auto Confirm User:一定要勾。不勾的話 Supabase 會等使用者 點驗證信,而你用的是收不到信的假網域,結果就是密碼明明對卻登不進去。

Email 要填什麼,取決於你想讓人在登入頁打什麼:

你想在登入頁輸入	Supabase 裡的 Email 就填	為什麼
sandyliu3056@gmail.com	sandyliu3056@gmail.com	輸入的字裡有 @,程式會原樣送出
geniqua_log	geniqua_log@ups-reprice.invalid	沒有 @ 的話,程式會自動補上 @ups-reprice.invalid 再送去驗證

.invalid 是網際網路標準保留下來、永遠不會存在的網域。用它當假信箱很安全:寄不出任何信,也絕對不會誤寄到別人真的信箱。代價是這種帳號不能用「忘記密碼」寄信重設——密碼要由你在後台改。

5 把主帳號設成管理者

左側 SQL Editor → New query → 貼上下面這段(email 換成你剛建的)→ 按 Run:

```
-- 把某個帳號設成主帳號(admin)
update auth.users
set raw_app_meta_data =
  coalesce(raw_app_meta_data, '{}'::jsonb) || '{"role":"admin"}'::jsonb
where email = 'sandyliu3056@gmail.com';

-- 跑完檢查一下,role 那一欄要出現 admin
select email, raw_app_meta_data->>'role' as role from auth.users;
```

為什麼要用 SQL、不能在畫面上點?因為角色一定要寫在 `app_metadata` 這個欄位,那一欄使用者自己改不動。另外一個叫 `user_metadata` 的欄位長得很像,但使用者自己就能改——角色寫在那裡的話,任何人都能把自己升成管理員。程式只認前者,這是刻意的。

其他所有帳號不用做任何事,預設就是一般使用者。

6 設定顯示名稱(選做)

不設的話,畫面右上角會直接顯示帳號。想顯示「Sandy Liu」就再跑一段:

```
update auth.users
set raw_user_meta_data =
  coalesce(raw_user_meta_data, '{} '::jsonb)
  || '{"display_name":"Sandy Liu"}'::jsonb
where email = 'sandyliu3056@gmail.com';
```

7 跑三支 SQL,把資料表建起來

在 SQL Editor 裡,把 repo 的這三個檔案整份貼上、各跑一次。順序沒有關係,三支都可以重複執行,跑第二次不會壞掉:

檔案	建了什麼	不跑會怎樣
supabase/schema.sql	profiles、user_settings	費率沒辦法存雲端,換電腦要重新匯入
supabase/login_history.sql	登入紀錄表 + 安全的寫入函式	Admin 頁的登入紀錄會顯示錯誤
supabase/local_settings.sql	本機模式的同步表與函式	你應該已經跑過了(現在的同步就是靠它)。切到 Supabase 模式後這支就用不到了,但留著沒關係

每一支跑完,畫面下方要出現 `Success. No rows returned` 之類的訊息。出現紅字就先別往下走,把錯誤訊息記下來。

8 部署兩支 Edge Function(建議做)

這一步是為了「以後在網站裡就能開帳號」和「UPS 地址驗證」。不做也能登入,只是加帳號還是得回 Supabase 後台點。

在你自己的電腦上(不是 Supabase 後台)開終端機:

```
npm install -g supabase
supabase login
supabase link --project-ref <你的 project ref,就是 URL 中間那串>

supabase functions deploy admin-users
supabase functions deploy ups-address

# 只有 ups-address 需要,填你的 UPS 開發者應用程式金鑰
supabase secrets set UPS_CLIENT_ID=xxx UPS_CLIENT_SECRET=xxx
```

部署完還有一個開關要動

到 Edge Functions → 點 admin-users → 設定裡把 Enforce JWT verification 關掉。

看起來反直覺,但這是對的:函式自己會驗身分,而且驗得更嚴——它會檢查來人的 `app_metadata.role` 是不是 admin,不是就直接 403。平台那層的預設驗證會擋掉正常的呼叫,反而讓 Admin 分頁出現「連不到函式」。

`ups-address` 部署完,把它的網址填進工具裡「帳單歷史」頁的 API 端點欄: `https://<你的專案>.supabase.co/functions/v1/ups-address`

9 設定 Site URL

Authentication → URL Configuration:

- Site URL 填你正式網站的網址。
- Redirect URLs 把預覽網址(Vercel 的 preview 之類)加進去。

只用帳號密碼登入的話這一步影響不大,但以後要開「忘記密碼」寄信、或用 Google 登入,信裡的連結就是照這裡設定的網址走的。

4 一步一步:架設帳號密碼(續)

10 改一個檔案,把開關打開

打開 `auth-config.js`(注意:是有連字號的那個), 改成這樣——重點是多了 `authMode` 這一行:

```
window.UPS_AUTH_CONFIG = {  
  authMode: "supabase",  
  url:      "https://你的專案.supabase.co",  
  anonKey:  "你的 anon / publishable key"  
};
```

就這樣。`index.html` 一個字都不用改——整套 Supabase 登入的程式碼 本來就在裡面等著,它只是在看這一行決定要走哪一邊。

要退回去也一樣簡單:把 `authMode` 那一行刪掉或改成別的值,就回到本機帳號模式。這個設計是刻意的——切換是「改設定」,不是「改程式」,所以隨時退得回來。

11 部署,然後驗證

把改好的 `auth-config.js` commit、push,讓主機重新部署。然後:

1. 開網站,看標題列右邊的 BUILD 日期。日期沒有跟著這次部署往前走,就是沒有拿到新檔案,不用再一個一個功能去找哪裡怪。先按 Ctrl+Shift+R(Mac 是 Cmd+Shift+R)強制重新整理。
2. 用第 4 步建的帳號密碼登入。
3. 登入後看右上角有沒有你的名字,以及 Admin 分頁在不在——在,表示第 5 步的管理者角色設對了。
4. 進 Admin 分頁,看得到帳號清單就表示第 8 步的函式也通了。

12 ▲ 切換前一定要知道的一件事

切過去之後,你的費率和帳單歷史會「看起來不見了」

原因:工具在存東西的時候,是用帳號 ID 當前綴分開存的,這樣同一台電腦換人登入才不會看到別人的合約價。而兩種模式的帳號 ID 不一樣:

- 本機模式 → `local::sandyliu3056@gmail.com`
- Supabase 模式 → 一組 UUID,像 `3f7a...-...-...`

ID 換了,前綴就換了,程式到新的前綴底下找,當然是空的。東西還在,只是在舊的名字底下。目前還沒有自動搬遷的程式碼。

另外還有一件事:本機模式那條「同步鑰匙」的雲端同步,切到 Supabase 模式後 就不再使用了(費率改存 `user_settings` 表)。帳單歷史在 Supabase 模式下目前沒有雲端同步,只留在各自的瀏覽器裡。

所以請從下面三個做法挑一個:

做法	怎麼做	評價
A 匯出再匯入	切換之前,在「設定」頁把費率設定匯出成檔案。切換完、用新帳號登入後,再匯入回去。	最穩,不用改任何程式。建議先用這個。
B 先試預覽站	先在 preview 網址上切,整套流程走一遍確認沒問題,再改正式站。	和 A 搭配著做最好
C 補搬遷程式	請人加一段:登入後如果新 ID 名下是空的,而這台電腦上有同名帳號的舊資料,就自動認領過來。	最漂亮,但要寫程式和測試

帳單逐筆明細(IndexedDB)同樣會換名字。如果那些期別已經同步到雲端,最保險的做法是切換前先確認每一期都歸檔過。

■ 出問題的時候:症狀對照表

畫面上看到的	真正的原因	怎麼修
「auth-config.js 沒有載到、或裡面是空的」	那個檔沒跟著部署,或 url / anonKey 沒填	確認它和 index.html 放在同一層。直接開你的網址/auth-config.js,看得到內容才算有上去
「supabase-js-2.112.3.js 沒有載到」	那個檔沒有一起上傳	直接開那個檔的網址,404 就是沒上傳
「Email 或密碼不正確」,但密碼你確定沒打錯	帳號建立時沒有勾 Auto Confirm	後台 Users 那一列看 email 是不是 confirmed。最快的做法是刪掉重建,這次記得勾

畫面上看到的	真正的原因	怎麼修
登得進去,但 Admin 分頁不見了	角色沒設,或設了但還在用舊的登入狀態	跑第 5 步的 SQL,然後登出再登入一次——角色是寫在登入當下拿到的 token 裡的,舊 token 還是舊角色
「連不到 admin-users 函式」	①函式沒部署 ②名稱不是剛好 <code>admin-users</code> ③JWT 驗證選項沒關	照第 8 步做一次。要分辨是哪一種,開瀏覽器 DevTools 的 Network 分頁看那一筆請求
登入紀錄頁顯示錯誤	<code>login_history.sql</code> 沒跑	回第 7 步
存設定時說「只存在這台電腦,雲端同步失敗」	SQL 沒跑完,或專案被暫停了	回第 7 步;順便到 Supabase 首頁看專案是不是 Paused
登入頁說「登入需要 https」	你是用 <code>http://</code> 開的	換成 https。瀏覽器的加密功能只在 https 或 localhost 才提供
部署完了,但功能還是舊的	快取,或主機根本沒部署到	網址後面加 <code>?v=1</code> 試;變新的就是快取問題;還是舊的就是主機那邊沒部署成功
登入完,費率和帳單歷史都空了	這不是壞掉,是第 12 步講的帳號 ID 換了	把切換前匯出的設定檔匯入回來

■ 上線前的檢查清單

- ☐ 公開註冊已經關掉(第 3 步)——這一項沒做等於門沒鎖
- ☐ `auth-config.js` 裡只有 URL 和 anon key,沒有 `service_role`
- ☐ pre-commit hook 裝好了:`git config core.hooksPath .githooks`
- ☐ 所有帳號都用了新密碼,沒有沿用曾經寫在程式裡的那組
- ☐ 三支 SQL 都跑過,沒有紅字
- ☐ 主帳號登入後看得到 Admin 分頁
- ☐ 用一個普通帳號登入試過,確認它看不到 Admin 分頁
- ☐ 切換前的費率設定已經匯出存好
- ☐ 部署後 BUILD 日期是新的

■ 以後要加一個人,怎麼做

切到 Supabase 之後,加人有兩條路,兩條都不用改程式、不用重新部署:

- 在網站裡:用主帳號登入 → Admin 分頁 → 新增帳號。(需要第 8 步的 `admin-users` 已經部署)
- 在 Supabase 後台:Authentication → Users → Add user,記得勾 Auto Confirm。

要停權一個人,就在後台把那個帳號刪掉或停用——他手上的登入狀態會過期,之後就再也進不來了。這是切到 Supabase 最實際的一個好處: 現在的做法得改 `index.html` 再部署一次,而且在那之前他還進得來。

最後,一句老實話

這份工具最有價值的東西——那六萬行的計價邏輯——不在 Supabase 裡,它在瀏覽器裡跑,也不會上傳到任何地方。Supabase 管的是「誰能進門」和「換台電腦東西還在不在」。

所以切換這件事沒有想像中可怕:弄壞了,把 `authMode` 那一行刪掉,就回到今天的樣子。